STRATHMORE UNIVERSITY

School of Computing and Engineering Sciences

Bachelor of Business Information Technology

BBT 2202

Advanced Object Oriented Programming

Project Report

InvenTrack

Inventory Management System

Field	Details
System Name	InvenTrack — Inventory Management System
Course	BBT 2202: Advanced Object-Oriented Programming
Institution	Strathmore University
School	School of Computing and Engineering Sciences
Technology Stack	Java, JavaFX, PostgreSQL, JDBC
Group Members	191900 Monica Michelle Njeri Mwangi 008331 Thomas Asamba

1. System Description

InvenTrack is a JavaFX-based desktop Inventory Management System developed as part of the BBT 2202 Advanced Object Oriented Programming project. The system provides a clean, intuitive interface for managing product stock in a real-world inventory setting, backed by a PostgreSQL relational database.

The system is designed to address three core operational needs:

- Adding new products with their name, quantity, and price
- Updating stock quantities for existing products
- Viewing all products with real-time out-of-stock highlighting

1.1 Project Requirements Compliance

Requirement	Implementation	Status
JavaFX GUI	Full sidebar navigation, TableView, forms, stats dashboard	Met
PostgreSQL via JDBC	DatabaseConnection (Singleton), ProductDAO (CRUD)	Met
Encapsulation	Private fields with getters/setters in Product class	Met
Inheritance	PerishableProduct and NonPerishableProduct extend Product	Met
Polymorphism	getProductInfo() overridden; updateStock() overloaded	Met
Generics	GenericDAO<T>, InputValidator generics, List<Product>	Met
Exception Handling	try-catch on all DB ops, custom input validation	Met
Input Validation (Bonus)	InputValidator utility with visual field feedback	Bonus
CSS Styling (Bonus)	Full styles.css with themed components	Bonus
TableView (Bonus)	Sortable, searchable TableView with inline actions	Bonus

2. OOP Design and Architecture

2.1 Project Structure

```
InventorySystem/
├── src/com/inventory/
│   ├── main/         MainApp.java          JavaFX entry point
│   ├── model/        Product.java           Abstract base class
│   │                 PerishableProduct.java  Extends Product
│   │                 NonPerishableProduct.java Extends Product
│   ├── db/           DatabaseConnection.java  Singleton JDBC manager
│   ├── dao/          GenericDAO.java          Generic interface
│   │                 ProductDAO.java           CRUD implementation
│   ├── controller/   InventoryController.java UI + business logic
│   └── util/         InputValidator.java       Validation utility
└── resources/        styles.css               JavaFX stylesheet
```

2.2 Encapsulation

The Product abstract class demonstrates encapsulation by declaring all fields as private and exposing them only through public getters and setters. This prevents external code from directly modifying internal state, ensuring data integrity throughout the application lifecycle.

```java
// Product.java — private fields with public accessors
public abstract class Product {
    private int id;
    private String name;
    private int quantity;
    private double price;

    public String getName()           { return name; }
    public void setName(String name)  { this.name = name; }
    public int getQuantity()          { return quantity; }
    public boolean isOutOfStock()     { return quantity == 0; }
}
```

2.3 Inheritance

Two concrete subclasses extend the abstract Product class. PerishableProduct adds expiry date tracking and expiry validation logic, while NonPerishableProduct provides stock valuation methods. This inheritance hierarchy enables code reuse and models the real-world distinction between product types.

```
// Inheritance hierarchy
public abstract class Product { ... }

public class PerishableProduct extends Product {
    // Adds: expiry date, isExpired(), getDaysUntilExpiry()
}

public class NonPerishableProduct extends Product {
    // Adds: getTotalStockValue()
}
```

2.4 Polymorphism

Polymorphism is demonstrated in two ways. First, the abstract getProductInfo() method is overridden in each subclass to return type-specific product descriptions. Second, the updateStock() method is overloaded with two signatures: one that adds to existing stock and one that replaces it entirely.

```
// Method Overriding — different output per subclass
@Override
public String getProductInfo() {   // PerishableProduct
    return "[Perishable] " + getName() + " | Expires: " + getExpiryDate();
}

// Method Overloading — same name, different signatures
public void updateStock(int amount)               // adds to existing
public void updateStock(int newQty, boolean replace)  // replaces entirely
```

2.5 Generics

Generics are applied in three areas: the GenericDAO<T> interface that defines type-safe CRUD operations, the validateNotNull<T> method in InputValidator, and throughout the application via typed collections such as List<Product> and ObservableList<Product>.

```
// GenericDAO<T> — type-safe data access interface
public interface GenericDAO<T> {
    void insert(T entity) throws SQLException;
    List<T> getAll() throws SQLException;
    T getById(int id) throws SQLException;
    void update(T entity) throws SQLException;
    void delete(int id) throws SQLException;
}
```

```
// ProductDAO implements GenericDAO<Product>
public class ProductDAO implements GenericDAO<Product> { ... }

// Generic null-check utility method
public static <T> boolean validateNotNull(T value) {
    return value != null;
}
```

3. Database Design

3.1 Entity Relationship

The system uses a single products table in PostgreSQL. The table is designed to support both product types (perishable and non-perishable) through the use of a nullable expiry_date column, avoiding the need for separate tables while maintaining data clarity.

3.2 Database Schema

```
-- PostgreSQL Schema: inventory_db
CREATE TABLE products (
    id          SERIAL PRIMARY KEY,
    name        VARCHAR(100)  NOT NULL,
    category    VARCHAR(50)   NOT NULL,
    quantity    INTEGER       NOT NULL DEFAULT 0,
    price       NUMERIC(10,2) NOT NULL,
    expiry_date DATE,                      -- NULL for non-perishable items
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3.3 JDBC Connection — Singleton Pattern

The DatabaseConnection class implements the Singleton design pattern, ensuring only one active database connection exists at any point during the application lifecycle. This prevents connection pool exhaustion and reduces overhead.

```
public class DatabaseConnection {
    private static Connection instance = null;

    public static Connection getConnection() throws SQLException {
        if (instance == null || instance.isClosed()) {
            Class.forName("org.postgresql.Driver");
            instance = DriverManager.getConnection(URL, USERNAME, PASSWORD);
        }
        return instance;
    }
}
```

3.4 Exception Handling in Database Operations

All database operations are wrapped in try-catch blocks that handle SQLException, ensuring the application degrades gracefully when database errors occur. The finally block is used to close resources, preventing memory leaks.

```
try (Connection conn = DatabaseConnection.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, product.getName());
    stmt.setInt(2, product.getQuantity());
    stmt.executeUpdate();
} catch (SQLException e) {
    System.err.println("DB Error: " + e.getMessage());
    throw e;  // Re-throw for UI layer to handle
}
```

4. System Features

4.1 Feature 1 — Add Product

The Add Product form captures the three core fields required by the project brief: product name, quantity, and price. Category is an optional field that defaults to Non-Perishable, preserving the OOP inheritance structure without adding friction to the basic use case.

Field	Type	Validation Rule
Product Name	Text (VARCHAR 100)	Required, non-empty, alphanumeric characters
Quantity	Integer	Required, must be 0 or greater
Price (KES)	Decimal	Required, must be greater than 0
Category	Dropdown	Optional, defaults to Non-Perishable
Expiry Date	Date Picker	Required only if Perishable is selected

4.2 Feature 2 — Update Stock

Stock can be updated in two ways. The Update Stock view provides a form with a dropdown that lists all products along with their current quantities. Out-of-stock products are displayed in red within the dropdown for easy identification. Additionally, each row in the product table has an inline Edit Stock button that opens a quick dialog for fast updates without navigating away from the table view.

4.3 Feature 3 — View All Products

The All Products view presents the complete inventory in a sortable, searchable TableView. The table displays the product name, category, quantity, price, and stock status for every product. Products with a quantity of zero have their name displayed in red text and their row highlighted with a red background, making out-of-stock items immediately identifiable at a glance.

> Key requirement met: When a product quantity is zero, the system displays its name in red. This is implemented via a custom TableCell renderer that applies -fx-text-fill: #dc2626 to the name column cell whenever the product's quantity equals zero.

4.4 Feature 4 — Dashboard Statistics

A live statistics bar above the product table displays four key metrics that update automatically whenever the data changes: total number of products, number of out-of-stock items, number of in-stock items, and the estimated total stock value calculated as the sum of (quantity x price) for all products.

5. User Interface Design

5.1 Layout Architecture

The application follows a sidebar navigation layout pattern. A dark sidebar (background: #0f172a) houses the navigation buttons and database status indicator. The main content area to the right contains the topbar, stats bar, and the active view. Three views are managed by a StackPane: All Products, Add Product, and Update Stock.

Component	JavaFX Class	Purpose
App shell	HBox	Splits sidebar and main area horizontally
Sidebar	VBox	Navigation, branding, DB status
Main area	VBox	Topbar + stats + content
Content switcher	StackPane	Holds all 3 views, toggles visibility
Product table	TableView<Product>	Displays all products with sorting
Add form	GridPane inside VBox	Two-column form for new products
Update form	VBox	Dropdown + quantity input
Stats bar	HBox of VBox cards	Four live metric cards

5.2 CSS Styling

The application uses an external CSS stylesheet (styles.css) loaded via scene.getStylesheets(). The stylesheet defines a consistent design system with a blue primary color palette (#1e40af), clean white surfaces, and semantic color coding for stock status.

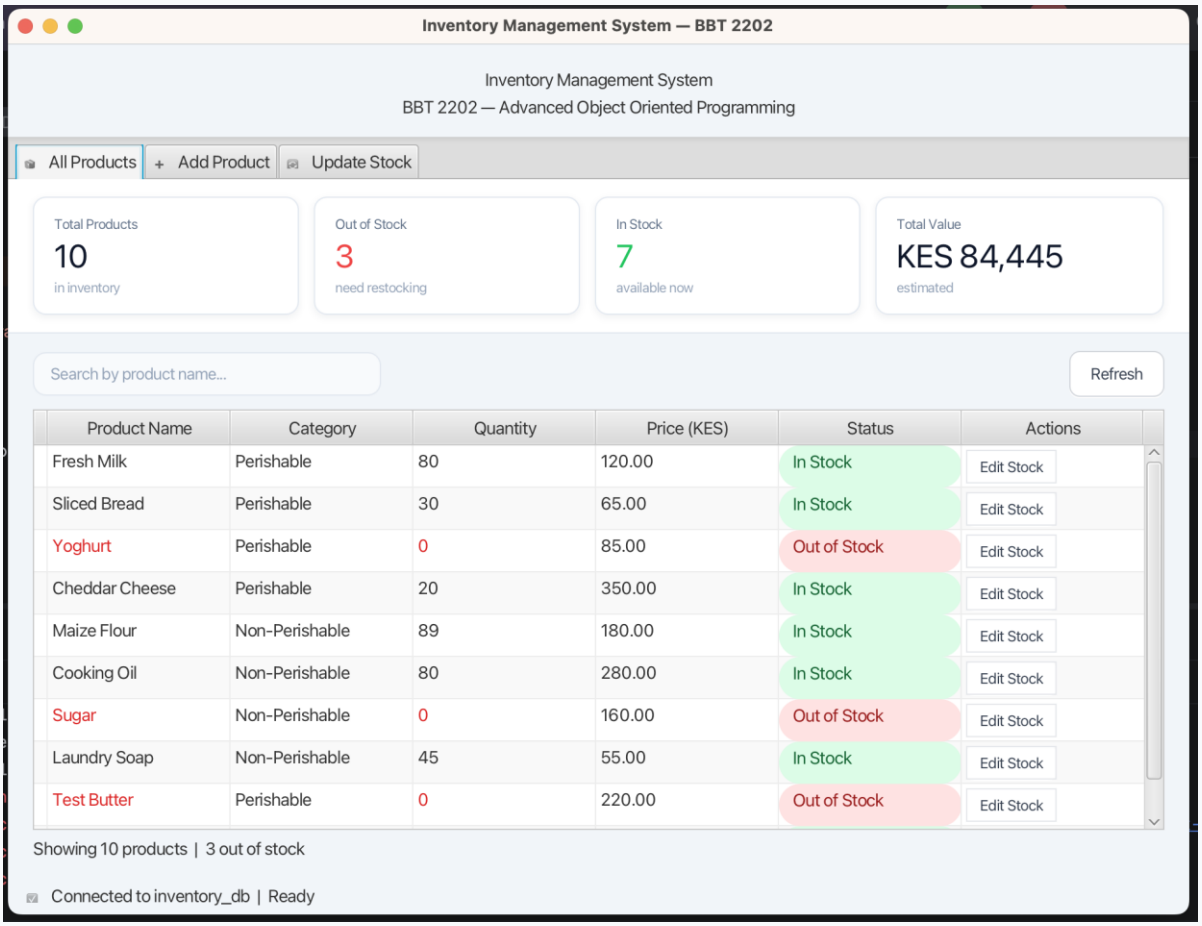
CSS Class	Applied To	Effect
.row-out-of-stock	TableRow	Red background (#fef2f2) on zero-stock rows
.btn-primary	Add/Update buttons	Blue background (#1e40af), white text
.form-input	All text fields	White background, gray border, blue focus ring
.stat-value-danger	OOS count label	Red text (#dc2626) for out-of-stock count
.stat-value-success	In-stock label	Green text (#16a34a)
.feedback-success	Form confirmation	Green-bordered success message box
.feedback-error	Form error	Red-bordered error message box

6. System Screenshots

The following screenshots demonstrate the system in operation. Please refer to the attached screenshots or the live demonstration during the class presentation.

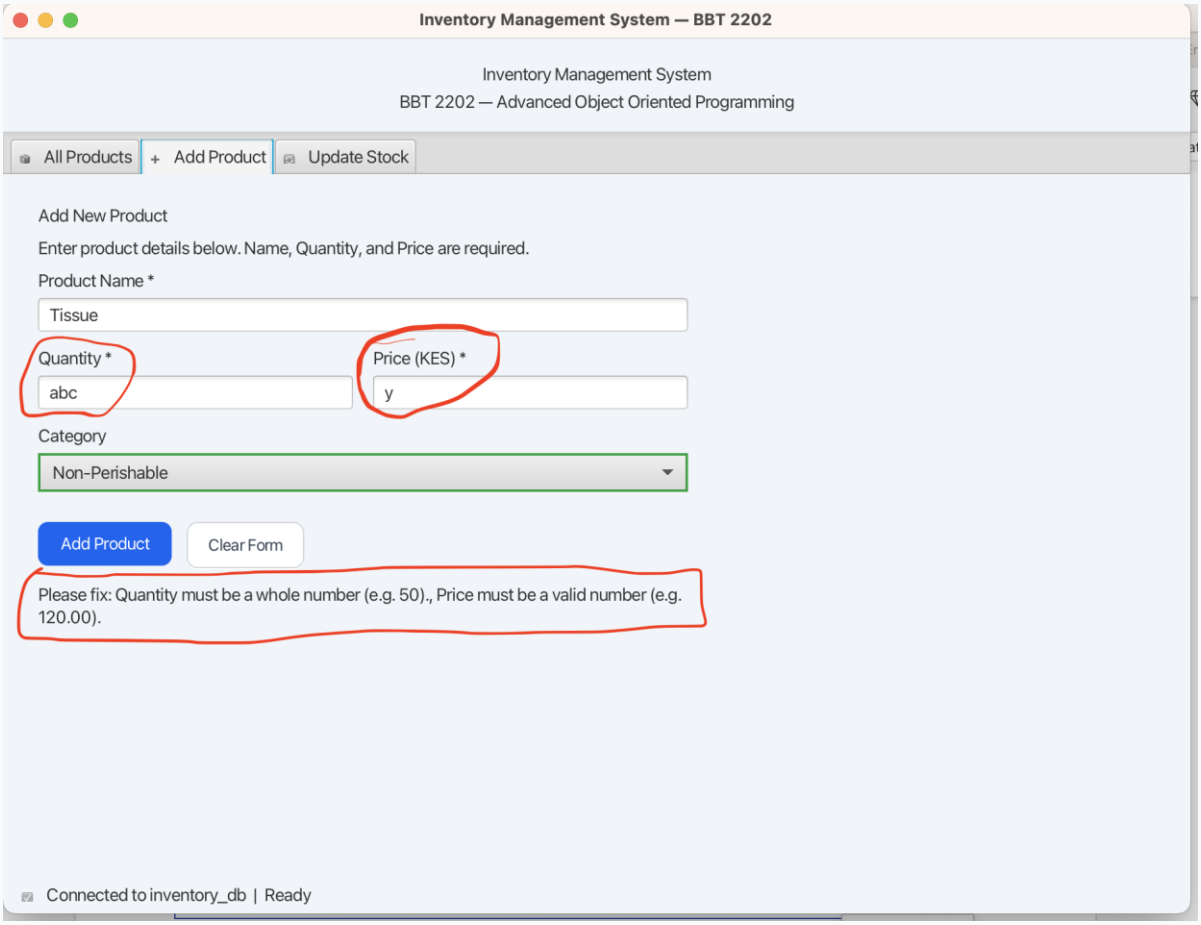
Screenshot 1

All Products View — showing product table with out-of-stock items highlighted in red



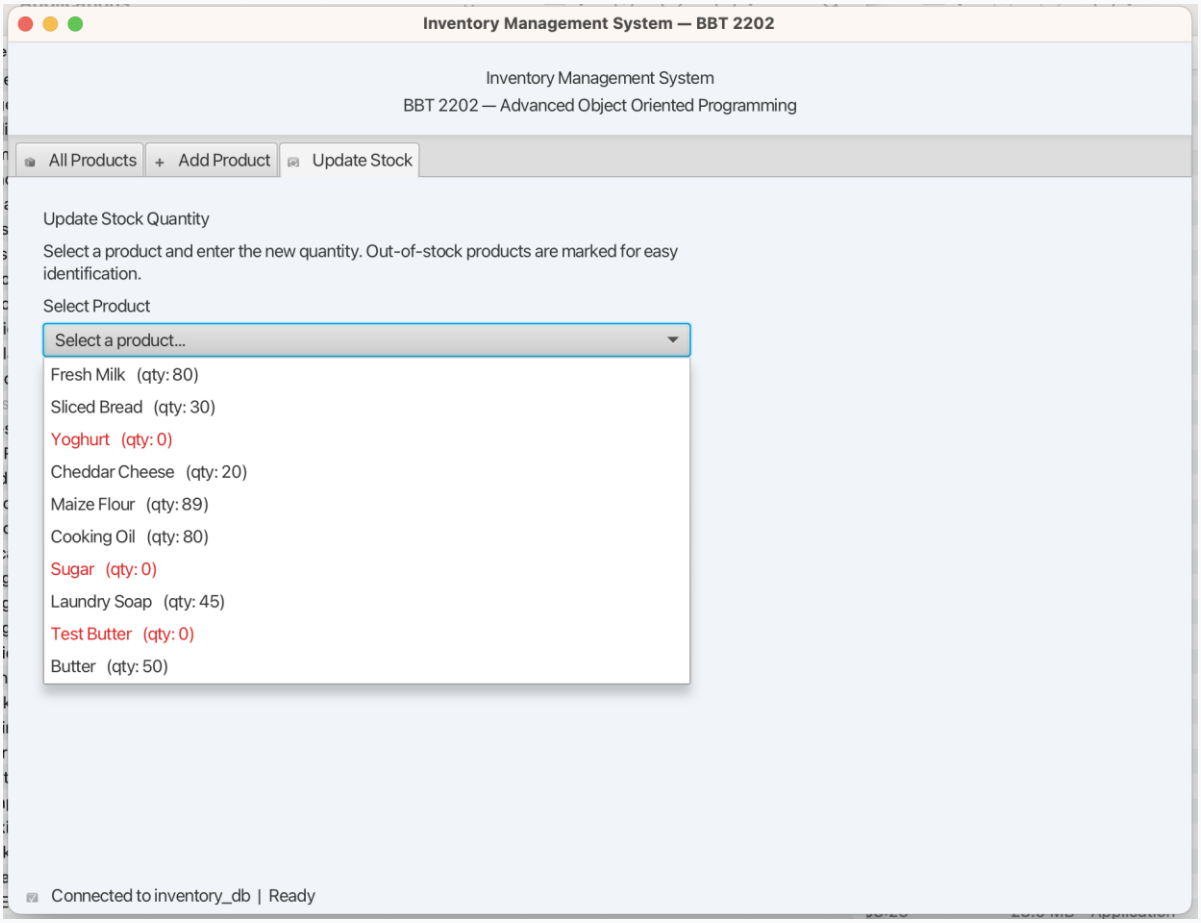
Screenshot 2

Add Product Form — showing form validation with error highlighting



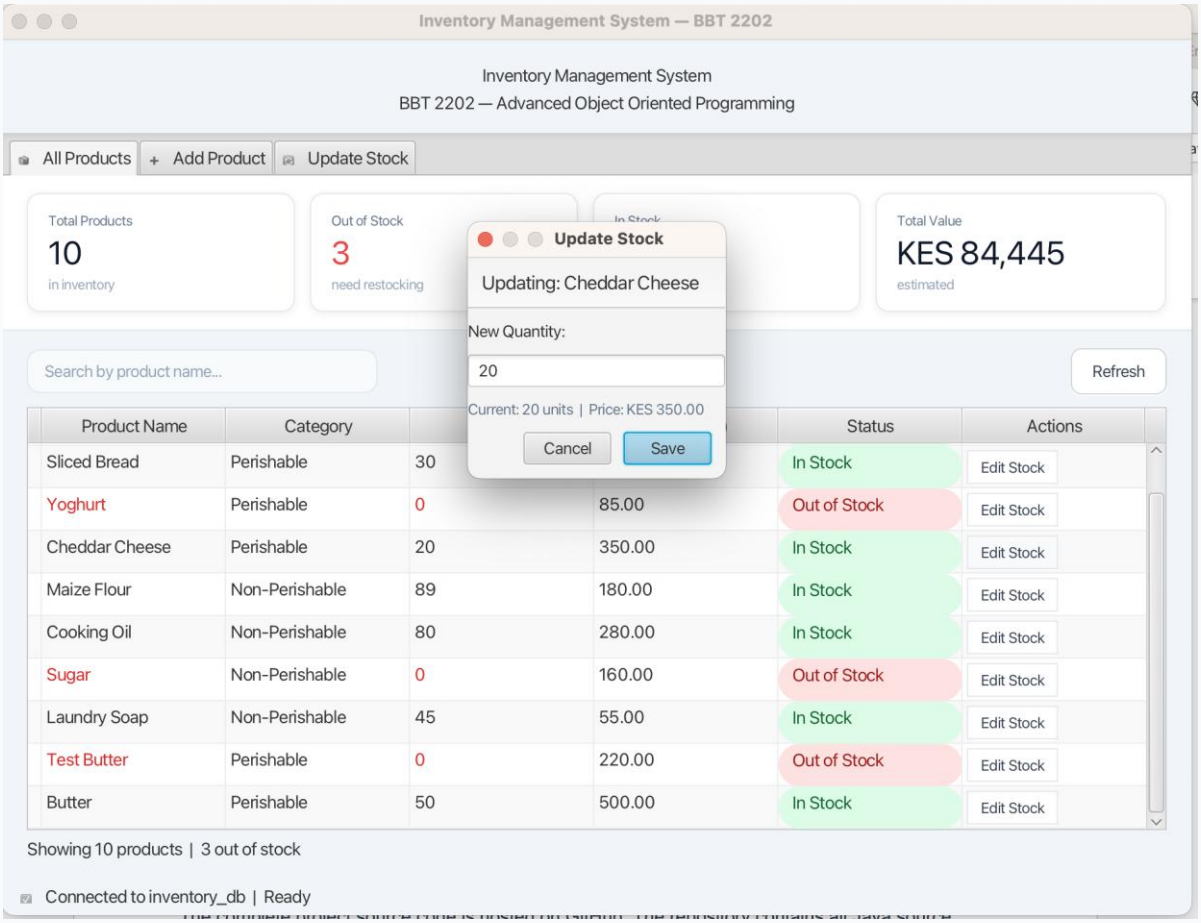
Screenshot 3

Update Stock View — showing dropdown with out-of-stock items marked in red



Screenshot 4

Quick Edit Dialog — inline stock update dialog from the table row



7. GitHub Repository

The complete project source code is hosted on GitHub. The repository contains all Java source files, the CSS stylesheet, the PostgreSQL setup script, and a detailed README with setup instructions.

Item	Details
Repository URL	https://github.com/thomasasamba-bot/InventorySystem
Branch	main
Source files	src/com/inventory/ (all packages)
Database script	database/inventory_db.sql
Stylesheet	resources/styles.css
README	README.md (setup + usage instructions)

8. Conclusion

InvenTrack successfully implements all core requirements of the BBT 2202 Advanced Object-Oriented Programming project. The system demonstrates the practical application of all five OOP principles in a real-world inventory management context:

1. Encapsulation is demonstrated through the private fields and accessor methods of the Product class hierarchy.
2. Inheritance is demonstrated through PerishableProduct and NonPerishableProduct, both extending the abstract Product class.
3. Polymorphism is demonstrated through method overriding of getProductInfo() and method overloading of updateStock().
4. Generics are demonstrated through the GenericDAO<T> interface, typed collections, and the generic validateNotNull<T> utility method.
5. Exception Handling is demonstrated through try-catch blocks on all database operations, custom validation logic, and user-facing error messages.

In addition to the core requirements, the system achieves all three bonus criteria: input validation with visual field feedback, CSS styling for a professional user interface, and an advanced TableView with inline actions and dynamic row highlighting.

> The system is ready for live demonstration during the class presentation session. All features are functional and connected to a live PostgreSQL database.